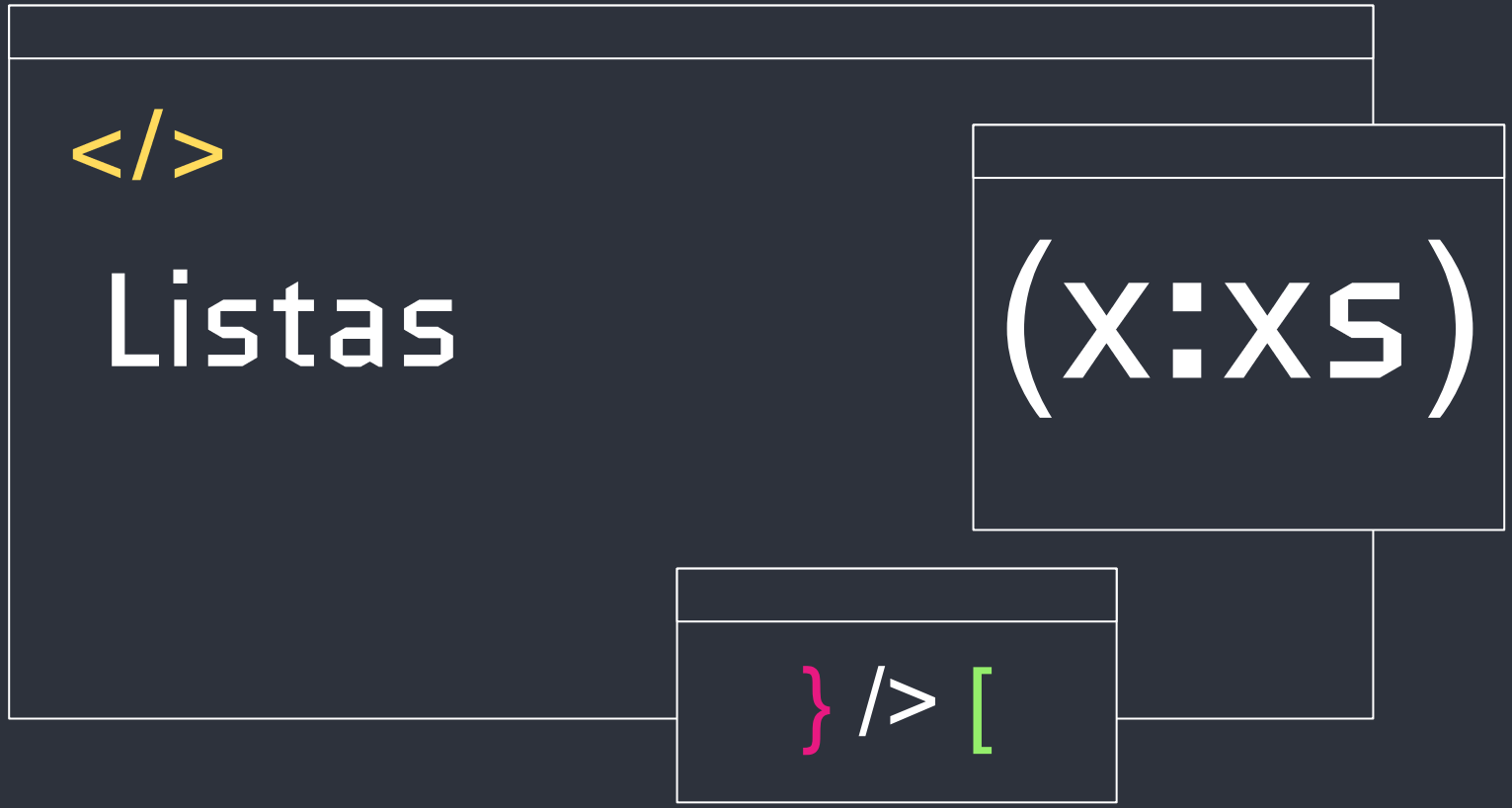


1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1 1 0 1


</>
Listas
(x:xs)
} /> [
1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 0 1

</Listas

Son conjuntos de elementos del mismo tipo, incluso puede haber listas de listas. Notación:

- Lista vacía [ ] - Ej.: [ ]
- Lista unitaria [X] - Ej.: [3]
- Lista simple xs o (x:xs) - Ej.: [2 5 6 7 8 9]

    x → cabeza xs → cuerpo

- Lista de listas xss o (xs:xss) - Ej:

    [[2] [2 4] [8 11]]

    xs → cabeza xss → cuerpo

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</Operaciones sobre listas

- Concatenación
[ ] : Xs = Xs : [ ] = Xs
(Xs :Ys) : Zs = Xs : (Ys : Zs)

- Longitud
#[ ] =0
(Xs:Ys)= #Xs + #Ys

- Head
hd(X:Xs)= X

- Tail
tl(X:Xs)= Xs

La concatenación recibe un elemento a izquierda y una lista a derecha
Ej:
3 : [2,4,1] → [3,2,4,1]
[3] : [[2,3],[7]] → [[3],[2,3],[7]]

Si se quisiera agregar el elemento al final se usa ++ convirtiendo el elemento a deseado en lista
Ej:
[2,4,1] ++ [3]→ [2,4,1,3]
[[2,3],[7]] ++ [[3]]→ [[2,3],[7],[3]]

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</Operaciones sobre listas

- map

```
cubo(x) = x*x*x
cuboL(x:xs) = map (cubo(x)) xs
```

- Filter

```
listaPares(x:xs) = filter( par(x)) xs
            where par(X)   | X mod 2=0 = True
                                        | otherwise = False
```

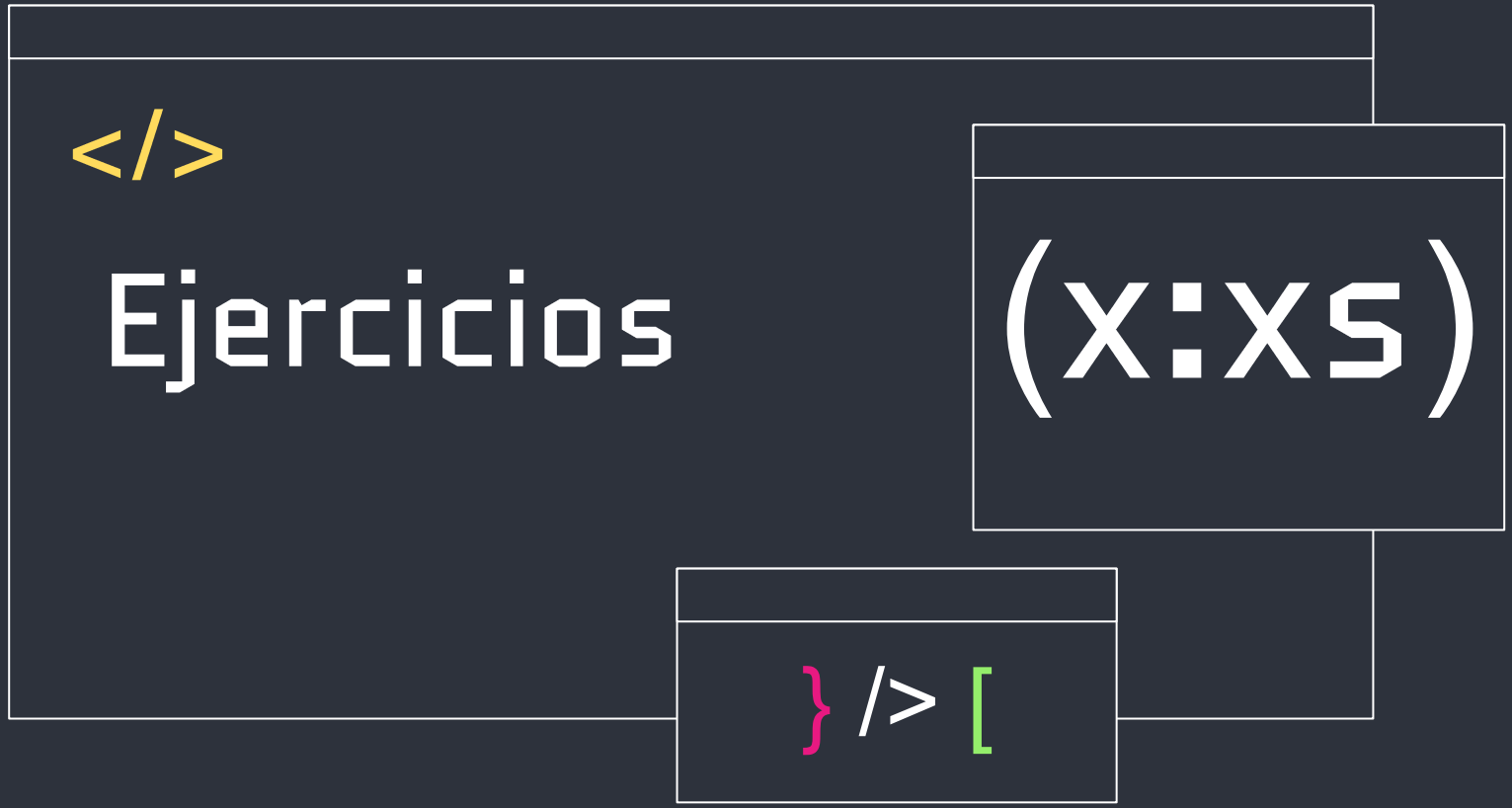
- take

```
take(0, [21 ,3 ,19]) -> [ ]
take(2, [21, 3, 19]) -> [ 21 3]
```

- takewhile

```
takewhile(par(X), [24, 6,3,25]) -> [ 24 6]
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1


</>
Ejercicios
(x:xs)
} /> [
1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</Ejercicios

Sumar todos los elementos de una lista

```
sumar::[Int]->Int
sumar([]) = 0
sumar(x:xs) = x + sumar xs
```

</Ejercicios

Contar todos los elementos de una lista

```
contar::[a] -> Int
contar(x:xs)=1 + contar xs
contar [] = 0
```

</Ejercicios

Contar las veces que aparece un número N en la lista

```
contar2::[Int] -> Int -> Int
contar2 (x:xs) k | x == k = 1 + contar2 xs k
                 | otherwise = contar2 xs k
contar2 [] k = 0
```

</Ejercicios

Determinar si en una lista de booleanos hay algún valor verdadero

```
algunv::[Bool] -> Bool
algunv(x:xs) | x = True
             | otherwise = algunv xs
algunv [] = False
```

</Ejercicios

Dada una lista de listas retornar una lista con la longitud de cada sublista (v1 Primitiva)

```
longLis::[[a]] -> [Int]
longLis [] = []
longLis(xs:xss) = length(xs):longLis xss
```

</Ejercicios

Dada una lista de listas retornar una lista con la longitud de cada sublista (v2 Definición local)

```
longLisv2::[[a]] -> [Int]
longLisv2 [] = []
longLisv2(xs:xss) = contar xs:longLisv2 xss
  where contar(x:xs) = 1 + contar xs
        contar [] = 0
```